

AWS IAM Governance Lab

Gobernanza de identidades y mínimo privilegio con Floci

Práctica de gobernanza de identidades · 30/10/2026

Índice

1. AWS IAM Governance Lab 3
1.1 Resultado 3
1.2 Contenido de la memoria 3
2. Entorno 4
2.1 Entorno Floci 4
2.2 Flujo de trabajo con Terraform 5
3. Usuarios y grupos 6
4. Políticas 7
4.1 Política de Dev-Team 7
4.2 Política de Ops-Team 8
4.3 Política de SecOps-Team 9
5. Auditoría con el simulador IAM 10
5.1 Resultados observados (selección) 10
6. Decisiones de seguridad y gobernanza 13
6.1 Por qué Terraform 13
6.2 Cómo se interpreta cada política JSON 13
6.3 Por qué se usa un Deny explícito 13
6.4 Grupos frente a permisos individuales 14
6.5 Infraestructura efímera para CI/CD 14
7. Conclusiones 15

1. AWS IAM Governance Lab

Memoria de la práctica de gobernanza de identidades para NextGen Solutions. La estructura IAM se ha definido en Terraform y desplegado en Floci: seis usuarios, tres grupos y permisos heredados exclusivamente por pertenencia a grupos. Las decisiones se han comprobado con `simulate-principal-policy`.

1.1 Resultado

El simulador confirmó los permisos previstos para `roberto`, `alicia`, `marta` y `camila`. En particular, `alicia` combina la lectura de auditoría con los permisos de desarrollo, pero `s3:DeleteObject` devuelve `explicitDeny`. Tras parar y reiniciar Floci sin persistencia, la lista de usuarios pasó de seis identidades a cero.

1.2 Contenido de la memoria

Sección	Contenido
Entorno Floci	Preparación, endpoint local y límite del emulador
Flujo de trabajo	Plan y despliegue Terraform, verificación y reinicio
Usuarios y grupos	Identidades, pertenencias y ausencia de políticas directas
Dev-Team	Lectura S3, operaciones DynamoDB y denegación de borrado
Ops-Team	Operaciones EC2 y gestión Lambda
SecOps-Team	Consulta para auditoría y protección frente a operaciones destructivas
Auditoría	Resultados reales del simulador IAM
Decisiones	Seguridad, gobernanza y ciclo de vida efímero
Conclusiones	Evaluación del diseño y límites

La infraestructura está en `envs/floci/` y `modules/iam-group/`; las políticas JSON en `policies/`. Los scripts de verificación están en `scripts/` y los datos íntegros obtenidos del emulador, en `evidence/`.

2. Entorno

2.1 Entorno Floci

Herramientas y conexión

La práctica se ejecutó en macOS con Terraform 1.17.0, Docker 28.3.3, AWS CLI 2.37.13 y Floci CLI 0.4.0. Floci arrancó un contenedor local accesible en `http://localhost:4566`. El proveedor AWS de Terraform usa credenciales ficticias `test/test` y dirige IAM y STS al endpoint local. Los scripts de verificación fijan también ese endpoint.

```
docker info
terraform version
aws --version
floci doctor
floci start
eval "$(floci env)"
aws iam list-users
```

`floci env` afecta solo a la terminal actual. Terraform no depende de esas variables: su proveedor declara el endpoint local y credenciales de prueba en `envs/floci/providers.tf`. El verificador usa `--endpoint-url http://localhost:4566` para evitar que una configuración AWS personal redirija las llamadas a la nube real.

Limitación de IAM en el emulador

El enunciado indica que Floci almacena políticas IAM, pero no aplica por defecto sus restricciones a las peticiones de servicio. Una operación de S3 hecha con las credenciales ficticias no demuestra el permiso efectivo de `roberto`. La evidencia de autorización procede de `aws iam simulate-principal-policy`, que evalúa las políticas de sus grupos.

Persistencia

La ejecución normal se hizo sin `--persist`. Al detener y arrancar el emulador se eliminó el contenedor anterior y desaparecieron las seis identidades. La salida íntegra está en `evidence/ephemeral.json`: `users_before` contiene los seis nombres y `users_after` es una lista vacía. El estado local de Terraform permanece, por lo que un nuevo `terraform apply` detecta los recursos ausentes y los reconstruye. Para conservar datos en Floci entre arranques se necesitaría configurar persistencia, lo que no forma parte de esta prueba.

2.2 Flujo de trabajo con Terraform

El entorno `envs/floci/` usa el proveedor oficial `hashicorp/aws` con credenciales ficticias y el endpoint IAM local. `modules/iam-group/` crea un grupo y le asocia su política inline JSON. En `locals.tf` se declaran los seis usuarios y sus pertenencias; `main.tf` crea los recursos. El archivo `.terraform.lock.hcl` fija la versión exacta del proveedor para repetir el despliegue.

Despliegue

Desde la raíz del repositorio, con Docker Desktop en funcionamiento:

```
floci start
terraform -chdir=envs/floci init
terraform -chdir=envs/floci fmt -check
terraform -chdir=envs/floci validate
terraform -chdir=envs/floci test
terraform -chdir=envs/floci plan
terraform -chdir=envs/floci apply
python3 scripts/verify.py
```

`bash scripts/deploy.sh` agrupa el `init` y el `apply` para repetir la creación. El primer plan mostró 18 recursos: seis usuarios, tres grupos, tres políticas inline y seis recursos de pertenencia (uno por usuario; `alicia` tiene dos grupos). Tras aplicar, un segundo `terraform plan -detailed-exitcode` terminó con «No changes».

`evidence/terraform-state.txt` lista los 18 recursos registrados en el estado de Terraform tras la reconstrucción.

`verify.py` contrasta el estado real con los JSON del repositorio y el simulador IAM. Guarda respuestas completas en `evidence/verification.json` y la comparación en `evidence/verification.md`. Para comprobar manualmente a `alicia`:

```
eval "$(floci env)"
aws iam list-groups-for-user --user-name alicia
arn="$(aws iam get-user --user-name alicia --query User.Arn --output text)"
aws iam simulate-principal-policy \
  --policy-source-arn "$arn" \
  --action-names s3:GetObject s3:DeleteObject cloudtrail:LookupEvents \
  --query 'EvaluationResults[].[EvalActionName,EvalDecision]' \
  --output table
```

Prueba de entorno efímero

Después de guardar las simulaciones, `bash scripts/check-ephemeral.sh` comprueba que el reinicio de Floci borra las identidades. `terraform -chdir=envs/floci plan` detecta los 18 recursos ausentes; `bash scripts/deploy.sh` los recrea y `python3 scripts/verify.py` vuelve a comprobarlos. El resultado del reinicio queda en `evidence/ephemeral.json`.

3. Usuarios y grupos

Grupo	Usuarios	Función
Dev-Team	roberto, sandra, alicia	Desarrollo sobre S3 y DynamoDB
Ops-Team	marta, alexis	Operación EC2 y Lambda
SecOps-Team	camila, alicia	Consulta y auditoría

Terraform crea los seis usuarios con `aws_iam_user`, los tres grupos con el módulo `iam-group` y las siete pertenencias con `aws_iam_user_group_membership`. La verificación contra Floci confirmó esas identidades y pertenencias. `alicia` aparece tanto en Dev-Team como en SecOps-Team. Cada grupo tiene una política inline propia. Se consultaron `list-user-policies` y `list-attached-user-policies` para los seis usuarios; ambas listas estaban vacías en todos los casos. Así se confirma que ningún permiso se asignó directamente a una persona.

La estructura permite cambiar permisos una sola vez en la política del grupo. Cuando alguien cambia de función, se modifica su pertenencia en vez de editar políticas individuales. La doble pertenencia de `alicia` es deliberada y se analiza en Decisiones.

4. Políticas

4.1 Política de Dev-Team

El archivo `policies/dev-team-policy.json` sigue el modelo del enunciado. Terraform lo lee desde `envs/floci/main.tf` y el módulo `iam-group` lo asocia al grupo como `DevTeamPolicy`.

Bloque	Efecto
`LecturaS3`	Permite `GetObject`, `ListBucket` y `ListAllMyBuckets`.
`CrudDynamoDB`	Permite leer, escribir, actualizar, borrar elementos, consultar y escanear tablas. No permite borrar tablas.
`DenegarBorradoS3`	Deniega explícitamente `DeleteObject` y `DeleteBucket` en todos los recursos.

En la simulación de `roberto`, `s3:GetObject` y `dynamodb:PutItem` resultaron `allowed`; `s3:DeleteObject` resultó `explicitDeny`. `ec2:StartInstances` resultó `implicitDeny`, lo que confirma que Dev-Team no recibe permisos de operación EC2.

La denegación explícita protege también a `alicia`, que pertenece a dos grupos. Si otro grupo llegara a permitir `s3:*`, el `Deny` seguiría prevaleciendo sobre ese `Allow`.

4.2 Política de Ops-Team

Terraform lee `policies/ops-team-policy.json` y el módulo `iam-group` la asocia a Ops-Team como `OpsTeamPolicy`.

policies/ops-team-policy.json

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ConsultarEC2",
      "Effect": "Allow",
      "Action": "ec2:Describe*",
      "Resource": "*"
    },
    {
      "Sid": "ArrancarYPararEC2",
      "Effect": "Allow",
      "Action": ["ec2:StartInstances", "ec2:StopInstances"],
      "Resource": "*"
    },
    {
      "Sid": "GestionarLambda",
      "Effect": "Allow",
      "Action": [
        "lambda:ListFunctions",
        "lambda:GetFunction",
        "lambda:CreateFunction",
        "lambda:UpdateFunctionCode",
        "lambda:UpdateFunctionConfiguration",
        "lambda:DeleteFunction",
        "lambda:InvokeFunction"
      ],
      "Resource": "*"
    }
  ]
}
```

Bloque	Efecto
`ConsultarEC2`	`ec2:Describe*` permite consultar instancias y otros datos EC2.
`ArrancarYPararEC2`	Permite `StartInstances` y `StopInstances`, sin conceder terminación.
`GestionarLambda`	Permite listar, consultar, crear, actualizar, borrar e invocar funciones Lambda mediante acciones enumeradas.

En la simulación de `marta`, `ec2:DescribeInstances`, `ec2:StartInstances`, `ec2:StopInstances`, `lambda:ListFunctions` y `lambda:CreateFunction` resultaron `allowed`. `s3:GetObject` dio `implicitDeny`.

Se enumeran las acciones de Lambda en vez de usar `lambda:*`, para evitar permisos adicionales no exigidos. El emulador usa `Resource: "*"` porque el enunciado no define ARNs concretos; en una cuenta real se acotarían instancias y funciones por ARN y se revisarían dependencias como `iam:PassRole` para crear funciones con un rol de ejecución.

4.3 Política de SecOps-Team

Terraform lee `policies/secops-team-policy.json` y el módulo `iam-group` la asocia a SecOps-Team como `SecOpsTeamPolicy`.

policies/secops-team-policy.json

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ConsultarCloudTrail",
      "Effect": "Allow",
      "Action": ["cloudtrail:LookupEvents", "cloudtrail:Describe*", "cloudtrail:Get*"],
      "Resource": "*"
    },
    {
      "Sid": "ConsultarCloudWatchLogs",
      "Effect": "Allow",
      "Action": ["logs:Describe*", "logs:Get*", "logs:FilterLogEvents"],
      "Resource": "*"
    },
    {
      "Sid": "ConsultarIAM",
      "Effect": "Allow",
      "Action": ["iam:Get*", "iam:List*"],
      "Resource": "*"
    },
    {
      "Sid": "ProtegerRecursosProduccion",
      "Effect": "Deny",
      "Action": ["ec2:TerminateInstances", "s3:DeleteObject", "dynamodb:DeleteTable"],
      "Resource": "*"
    }
  ]
}
```

Bloque	Efecto
`ConsultarCloudTrail`	Permite `LookupEvents`, `Describe*` y `Get*` de CloudTrail.
`ConsultarCloudWatchLogs`	Permite `Describe*`, `Get*` y `FilterLogEvents` de CloudWatch Logs.
`ConsultarIAM`	Permite `Get*` y `List*` de IAM para inspeccionar la configuración.
`ProtegerRecursosProduccion`	Deniega `ec2:TerminateInstances`, `s3:DeleteObject` y `dynamodb:DeleteTable`.

El `Deny` se aplica a `Resource: "*"`: protege también los recursos de producción y, de forma deliberadamente más estricta, impide esas tres operaciones sobre cualquier recurso. Así seguirá prevaleciendo si en el futuro otro grupo concede permisos amplios. Para limitarlo solo a producción en AWS real harían falta nombres o etiquetas de recursos definidos y condiciones adecuadas a cada servicio.

5. Auditoría con el simulador IAM

Terraform creó los usuarios, grupos, pertenencias y políticas en Floci. El enunciado advierte que Floci no aplica por defecto las políticas al invocar los servicios. Por eso se usó `aws iam simulate-principal-policy` sobre el ARN de cada usuario. El simulador incluyó las políticas de todos sus grupos; no ejecutó las acciones sobre recursos reales.

5.1 Resultados observados (selección)

Usuario	Acción	Decisión
roberto	`s3:GetObject`	`allowed`
roberto	`s3:DeleteObject`	`explicitDeny`
roberto	`dynamodb:PutItem`	`allowed`
alicia	`s3:GetObject`	`allowed`
alicia	`s3:DeleteObject`	`explicitDeny`
alicia	`cloudtrail:LookupEvents`	`allowed`
marta	`ec2:DescribeInstances`	`allowed`
marta	`ec2:StartInstances`	`allowed`
marta	`lambda:CreateFunction`	`allowed`
marta	`s3:GetObject`	`implicitDeny`
camila	`cloudtrail:LookupEvents`	`allowed`
camila	`logs:FilterLogEvents`	`allowed`
camila	`s3:DeleteObject`	`explicitDeny`
camila	`dynamodb:PutItem`	`implicitDeny`

Estos resultados se obtuvieron el 30 de octubre de 2026. El archivo `evidence/verification.json` conserva la respuesta íntegra de Floci, incluidas las evaluaciones por acción; `evidence/verification.md` presenta la comparación entre valor esperado y obtenido. La comprobación automatizada también verificó las siete pertenencias, la coincidencia de las tres políticas aplicadas con los JSON del repositorio y la ausencia de políticas directas en usuarios.

`allowed` significa que existe un `Allow` aplicable; `implicitDeny`, que ninguna política concede la acción; `explicitDeny`, que una regla `Deny` aplicable prevalece sobre cualquier permiso concedido.

```
Floci IAM simulator  |  roberto
ARN: arn:aws:iam::000000000000:user/roberto
Fecha UTC: 2026-10-30T10:35:53.833394+00:00

ACCION                          DECISION
s3:GetObject                    allowed
s3:DeleteObject                 explicitDeny
dynamodb:PutItem                allowed
ec2:StartInstances              implicitDeny
```

```
Floci IAM simulator  |  alicia
ARN: arn:aws:iam::000000000000:user/alicia
Fecha UTC: 2026-10-30T10:35:53.833394+00:00

ACCION                          DECISION
s3:GetObject                    allowed
s3:DeleteObject                 explicitDeny
dynamodb:PutItem                allowed
cloudtrail:LookupEvents         allowed
iam:ListUsers                   allowed
ec2:TerminateInstances          explicitDeny
dynamodb:DeleteTable            explicitDeny
```

```
Floci IAM simulator  |  marta
ARN: arn:aws:iam::000000000000:user/marta
Fecha UTC: 2026-10-30T10:35:53.833394+00:00

ACCION                          DECISION
ec2:DescribeInstances           allowed
ec2:StartInstances              allowed
ec2:StopInstances               allowed
lambda:ListFunctions            allowed
lambda:CreateFunction           allowed
s3:GetObject                    implicitDeny
```

```
Floci IAM simulator  |  camila
ARN: arn:aws:iam::000000000000:user/camila
Fecha UTC: 2026-10-30T10:35:53.833394+00:00

ACCION                          DECISION
cloudtrail:LookupEvents         allowed
logs:FilterLogEvents            allowed
iam:ListUsers                   allowed
s3:DeleteObject                 explicitDeny
ec2:TerminateInstances          explicitDeny
dynamodb:PutItem                implicitDeny
```

6. Decisiones de seguridad y gobernanza

6.1 Por qué Terraform

La configuración de `envs/floci/` declara de forma revisable las identidades, sus pertenencias y las políticas de grupo. El módulo `modules/iam-group/` evita repetir la creación de cada grupo y la asociación de su JSON. `terraform plan` muestra el cambio antes de aplicarlo; tras el primer despliegue, el plan devolvió «No changes». Esto permite reconstruir exactamente el laboratorio después de un reinicio efímero de Floci. El estado Terraform se guarda localmente y se excluye de Git; `.terraform.lock.hcl` fija la versión del proveedor.

6.2 Cómo se interpreta cada política JSON

`Version` identifica la versión del lenguaje de políticas de IAM, aquí `2012-10-17`; no es la fecha de creación del archivo. `Statement` contiene las reglas evaluables. `Sid` da un nombre a cada regla para reconocer su intención. `Effect` indica `Allow` o `Deny`. `Action` enumera operaciones AWS, con comodines cuando se pide una familia de acciones. `Resource` determina a qué recursos se aplica cada regla.

En esta práctica el enunciado no proporciona ARNs de buckets, tablas, instancias o funciones; por eso las políticas usan `Resource: "*"`. El mínimo privilegio se aproxima restringiendo las acciones por grupo. Antes de pasar el diseño a una cuenta AWS real habría que acotar recursos y revisar cada comodín.

6.3 Por qué se usa un Deny explícito

La ausencia de `s3:DeleteObject` en los `Allow` de Dev-Team produciría solo `implicitDeny`. Si en el futuro otra política de cualquiera de los grupos de `alicia` concediera `s3:*`, ese permiso permitiría borrar objetos. En cambio, el `Deny` explícito de Dev-Team prevalece aunque aparezca otro `Allow`. La simulación actual de `alicia` confirmó `explicitDeny` para `s3:DeleteObject`.

SecOps-Team también deniega explícitamente tres operaciones destructivas. Como `alicia` pertenece a ese grupo, recibe ambas protecciones. El bloqueo de SecOps usa `Resource: "*"`, más amplio que «producción»; se eligió así porque el ejercicio no identifica recursos de producción concretos.

6.4 Grupos frente a permisos individuales

Con grupos, una política expresa la función de un equipo y se modifica una sola vez. Incorporar a una persona consiste en añadirla al grupo; retirar una función consiste en quitar esa pertenencia. Esto facilita la revisión de quién tiene cada capacidad y reduce excepciones difíciles de encontrar.

Con permisos asignados usuario por usuario, un cambio de función obliga a localizar y editar cada identidad. Para 10 empleados todavía podría revisarse manualmente; con 100 aumenta la probabilidad de diferencias y permisos olvidados; con 1000 el coste operativo y de auditoría hace necesario automatizar las altas, bajas y revisiones sobre grupos o roles. La doble pertenencia de `alicia` muestra cómo combinar funciones sin duplicar políticas.

6.5 Infraestructura efímera para CI/CD

La comprobación real registró seis usuarios creados por Terraform antes de `floci stop` y ninguno después de `floci start`, sin persistencia configurada. El estado local de Terraform permaneció; el plan detectó 18 recursos ausentes y `apply` los recreó. Cada ejecución de una tubería CI/CD puede partir de un estado vacío, aplicar la configuración, probarla y descartarla al finalizar. Esto evita que un test dependa de datos residuales de otro y hace reproducible el entorno. Para conservar estado en Floci entre arranques, habría que elegir explícitamente un modo persistente.

7. Conclusiones

Terraform desplegó y verificó en Floci seis usuarios, tres grupos, siete pertenencias y tres políticas inline asociadas solo a grupos. El simulador confirmó permisos de desarrollo, operaciones y auditoría, así como denegaciones implícitas para acciones ajenas a cada función.

El caso de `alicia` demuestra la precedencia del `Deny` explícito: recibe permisos de Dev-Team y SecOps-Team, pero no puede borrar objetos S3 según la simulación. La gobernanza por grupos permite mantener y auditar esos permisos sin configuraciones individuales.

El reinicio sin persistencia borró las seis identidades. Terraform detectó y recreó los 18 recursos declarados, lo que confirma que el entorno puede reconstruirse desde código y utilizarse como base limpia para pruebas CI/CD. El simulador verifica decisiones de autorización; una comprobación en AWS real exigiría además revisar ARNs, dependencias entre servicios y restricciones adicionales del entorno de producción.